Coursework 1: Artificial Neural Networks

Machine Learning

Check coursework deadline on Scientia

1 Overview

Please read this manual thoroughly as it contains useful information about the assignment, as well as the support provided.

In this assignment, you will learn to implement and train **neural network models** and apply them to solve a regression task. The assessment of this coursework is based on your code submission, and will be evaluated using `LabTS` tests. A written report is **not** required. You should submit your code via `LabTS`, and ensure that your submission has been recognised by Scientia.

This assignment is split into 2 parts:

- **Part 1: Create a neural network mini-library**. You will create a low-level implementation of a multi-layered neural network using `NumPy`, including a basic implementation of the backpropagation algorithm. The task also involves implementing necessary functions for data preprocessing.
- **Part 2: Create and train a neural network for regression**. You will train and evaluate a neural network model to predict house values using the California House Prices dataset.

The initial skeleton code for both parts is provided in your Gitlab repos. Your task is to complete the missing parts, which are clearly indicated in the python files. You can test your code on LabTS during development and the LabTS results will be used for assessment also.

The mini-library in Part 1 must be implemented using `NumPy`. For this part, you may not use libraries that implement automatic differentiation, such as `PyTorch` or `TensorFlow`, but you may use basic python libraries (`os`, `sys`, `math`, etc.).

For Part 2 you must use the widely-used `PyTorch` neural network library to train a model. For this part, the libraries you can use are only limited by what will run successfully on LabTS.

2 Where to get help?

- **Lab sessions** run every week where you'll find very helpful Teaching Assistants. Beyond the coursework they also have a very good understanding of how ML works so please reach out. The helpers will be in the main labs area, usually near the printers.
- **EdStem** is the online platform we use for question and answers. It is great for getting help from your peers too.
- **Office Hours** are usually offered by the main lecturer. Please check with them when and where they hold office hours.

3 Group formation

This is a group project **of up to 4 students**. You'll form your groups via Scientia. **Only the group leader needs to submit** and the marks will be shared with all the members. We strongly recommend you work in groups of 4 to help each other.

1. Each student has a GitLab repository based on their username `https://gitlab.doc.ic.ac.uk/lab2627_autumn/Neural_Networks_[username]`.
2. If you don't see this repository make sure you have selected this module, we cannot manually add students as the registration is centrally managed. The repositories are created by us, so if you join the module later, you need to let us know.
3. As the group leader, you can add others to your GitLab repository to work together.
4. Any unused repositories will be ignored so don't worry. For example, if your group leader submitted, your repository is ignored even if you have code there.
5. You commit and push your code to then test on LabTS (see section below).

The source of truth is Scientia. If you are part of a group, make sure your leader has submitted via LabTS and has tested their submission. The feedback and marking will be coordinated via Scientia, so it is really important. It is your responsibility to coordinate and ensure there is a linked submission.

Can I change my group? Yes, you can change your groups but please ensure to use the given the repositories. Do not create or delete them until the exercise is done. For example, if you leave a group and start a new one as the leader, then you need to submit using your specific repo. If you join another group which has space, then their leader submits.

4 Generative AI Recommendation

For Part 1, we strongly recommend you attempt to implement the code segments yourself. These are usually only one or couple of lines of code. Arriving at the solution will really help your understanding of how neural networks function behind the scenes.

For Part 2, we recommend brainstorming and attempting the key parts yourself. The objective is to see how a full machine learning process works from loading and processing data to training the model. Again, the amount of coding required is minimal but gives you a chance to go behind the scenes of machine learning.

5 Guidelines and setup

The link to your group GitLab repository with the skeleton code will be available on `LabTS` (https://teaching.doc.ic.ac.uk/labts/). We recommend using Git to manage your code and keep backups of different code versions. You can find a quick introductory handbook to Git in the GitHub documentation: https://guides.github.com/introduction/git-handbook/.

You should implement all your code in **Python 3**, we will not provide support for other languages. Your code will be assessed and tested on `LabTS`. The `python` environment on the `LabTS` servers closely mimics the ones on the lab workstations. This is a good reason for you to work on the lab workstations! You can log into the lab machines remotely using ssh. For setting up a python environment that matches the LabTS environment, please refer to the previous tutorials on Scientia. Dependencies can be installed using the provided `requirements.txt`, or on the lab workstations you can activate the pre-configured environment with `source /vol/lab/ml/mlvenv/bin/activate`.

You are able to test your code prior to submission using `LabTS`. To do this, push to your GitLab repository and access the tests through the `LabTS` portal. On `LabTS` you will be able to see the list of all your commits, along with a "Request Test" button which will run the tests on that particular commit and generate a pdf report showing which tests you passed or failed. We highly recommend that you test your code on `LabTS` as soon as possible and throughout the project, to avoid discovering incompatibility issues with the test environment at the last minute. You can test your code on `LabTS` as often as you need to.

When you are ready, **submit** your desired commit via `LabTS`, and ensure that your GitLab Hash has been recognised by Scientia. Only your submitted commit will be considered for marking.

Part 1: Create a neural network mini-library

In this part, you will implement your own modular neural network mini-library using `NumPy`. This will require you to complete a number of classes in `part1_nn_lib.py`. The aim of this part is to familiarise you with the forward and backward passes of neural network components, as well as parameter updates using gradient descent, and simple data preprocessing functionality.

A simple dataset (https://en.wikipedia.org/wiki/Iris_flower_data_set, given in the Gitlab repository as `iris.dat`) and sample code demonstrating intended usage is provided for debugging purposes as you work through this part of the coursework.

The expected behaviour, inputs, outputs, and array shapes are described for each method in the **docstrings in the code**.

Q1.1: Implement a linear layer

You need to implement a linear layer (which performs an affine transformation $XW + B$ on a batch of inputs X) by completing the missing parts of the `LinearLayer` class.

Note that for the `forward` and `backward` methods, you should not use Python loops (for efficiency reasons); use vectorized operations instead.

Here is an example of how this class can be used:

```
layer = LinearLayer(n_in=3, n_out=42)
# `inputs` shape: (batch_size, 3)
# `outputs` shape: (batch_size, 42)
outputs = layer(inputs)
# `grad_loss_wrt_outputs` shape: (batch_size, 42)
# `grad_loss_wrt_inputs` shape: (batch_size, 3)
grad_loss_wrt_inputs = layer.backward(grad_loss_wrt_outputs)
layer.update_params(learning_rate)
```

Q1.2: Implement activation function classes

You need to implement the missing parts of the `SigmoidLayer` and `ReluLayer` activation function classes (note: linear activation can be achieved without an activation class).

Note that for the `forward` and `backward` methods, you should not use Python loops (for efficiency reasons); use vectorized operations instead.

Q1.3: Implement a multi-layer network

You need to implement a multi-layer network (consisting of stacked linear layers and activation functions) by completing the missing parts of the `MultiLayerNetwork` class. Your implementation should support the activations `"sigmoid"`, `"relu"`, and `"identity"` (a linear passthrough without an activation layer).

Here is an example of how this class can be used:

```
# The following command will create a MultiLayerNetwork object
# consisting of the following stack of layers:
#   - LinearLayer(4, 16)
#   - ReluLayer()
#   - LinearLayer(16, 2)
#   - SigmoidLayer()
network = MultiLayerNetwork(
    input_dim=4, neurons=[16, 2], activations=["relu", "sigmoid"]
)
# `inputs` shape: (batch_size, 4)
```

```
# `outputs` shape: (batch_size, 2)
outputs = network(inputs)
# `grad_loss_wrt_outputs` shape: (batch_size, 2)
# `grad_loss_wrt_inputs` shape: (batch_size, 4)
grad_loss_wrt_inputs = network.backward(grad_loss_wrt_outputs)
network.update_params(learning_rate)
```

Q1.4: Implement a preprocessor

Data normalization can be crucial for effectively training neural networks. For this question, you will need to implement a `Preprocessor` class which performs min-max scaling such that the data is scaled to lie in the interval $[0, 1]$. Same as before, complete the methods of the `Preprocessor` class.

- **Constructor:** Initialize the Preprocessor according to the provided dataset. This function does **not** modify the provided dataset.
- **Apply method:** Complete the `apply` method such that it applies the pre-processing operations to the provided dataset and returns the preprocessed version.
- **Revert method:** Complete the `revert` method such that it reverts the pre-processing operations that have been applied to the provided dataset and returns the reverted dataset. For any dataset `A`, `prep.revert(prep.apply(A))` should return `A`.

Here is an example of how this class can be used:

```
prep = Preprocessor(dataset)
normalized_dataset = prep.apply(dataset)
original_dataset = prep.revert(normalized_dataset)
```

Part 2: Create and train a neural network for regression

In this part, you will work on the California House Prices Dataset.[1] The dataset covers all the block groups in California from the 1990 Census. A block group is the smallest geographical unit for which the US Census Bureau publishes sample data, and on average it includes 1425.5 individuals living in a geographically compact area. The dataset contains ten variables:

1. **longitude:** longitude of the block group
2. **latitude:** latitude of the block group
3. **housing_median_age:** median age of the individuals living in the block group

[1] This data was initially featured in the following paper: *Pace, R. Kelley, and Ronald Barry. "Sparse spatial autoregressions." Statistics & Probability Letters 33.3 (1997): 291-297.*

4. **total_rooms:** total number of rooms in the block group
5. **total_bedrooms:** total number of bedrooms in the block group
6. **population:** total population of the block group
7. **households:** number of households in the block group
8. **median_income:** median income of the households comprise in the block group
9. **ocean_proximity:** proximity to the ocean of the block group
10. **median_house_value:** median value of the houses of the block group

In this part of the coursework, you will implement a neural network architecture to infer the median house value of a block group from the value of all other attributes.

You can find the skeleton code for this part in `part2_house_value_regression.py`. It defines a `Regressor` class that you will need to complete by implementing the `__init__`, `fit`, `predict`, and `evaluate` methods. This file also defines two functions `save_regressor` and `load_regressor`, to save and load a `Regressor` model using `pickle` functions. You can modify these functions if you need to but make sure to change both of them so they can work in tandem. In this part, you must use the `PyTorch` neural network library. You can find an introductory tutorial for `PyTorch` in the library documentation (https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html). You are also allowed to use other helpful libraries that run on LabTS, such as scikit-learn.

You can find the data for this part of the coursework in the file `housing.csv`. As the data is more complex than in the first part, corresponding helper methods (`_preprocessor` and `_revert_preprocessor`) are provided on the `Regressor` class, which one-hot encode textual entries, replace missing numerical values using medians computed on the training data, and perform min-max scaling to normalise the data and revert target scaling. To read this `csv` file into python3, we use the function `read_csv` from the `pandas` library, as it allows to handle multi-datatype. `pandas` is a widely-used Python library for data-sciences. It offers a nice interface to handle and visualise large data tables. You can find a quick introduction to `pandas` here: https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html. Note that the data arguments given during `LabTS` tests for Part 2 will be in the format of `pandas` DataFrame objects.

The expected behaviour, inputs, and outputs are described for each method in the **docstrings in the code**.

Here is an example of how the `Regressor` class can be used to train a regressor model on the data:

```
# Assuming x_train contains the data and y_train the corresponding targets
regressor = Regressor(x_train, nb_epoch = 10)
regressor.fit(x_train, y_train)
save_regressor(regressor)
```

Feel free to define any supplementary methods or classes you may find necessary. However, don't change the names of existing functions or classes, as they will be called by LabTS during

testing. Also, if you add additional arguments to functions, make sure they are optional and include a default value, so the function still works with the original set of arguments.

Deliverables

On Scientia, you will need to submit the GitLab hash via LabTS corresponding to the chosen commit on your GitLab repository. The code and model in this commit will then be tested and assessed.

In this repository, you should have completed and pushed:

- Completed version of `part1_nn_lib.py`.
- Completed version of `part2_house_value_regression.py`.
- Committed and pushed your best model as `part2_model.pickle`. This has to be saved by `save_regressor` and it will be loaded during testing using `load_regressor`.

6 Important Notes

1. **Important!** Make sure to evaluate on `LabTS` early, to ensure that your code runs there correctly. You will lose points on any tests where the code fails or does not run.
2. Make sure to have at least one working commit tested on `LabTS` a few days before the deadline. The `LabTS` queues for running tests will likely get long as the deadline approaches and your tests might not complete on time if you don't submit them early enough.
3. Do not rename the provided classes, functions or methods. If you add any input parameters to a provided function/method, make sure you provide a default value for them. `LabTS` expects the functions/methods/classes to be used in the way provided; you may fail the `LabTS` tests otherwise. You can add your own classes/methods/functions as needed, as long as you do not change the expected signature of those that are provided.
4. Feel free to ask the GTAs in the lab sessions for advice, they are happy to help. However, please don't ask them to debug your code. For this, I would recommend 1) constructing your own local tests to check that the code does what it should be doing, and 2) adding some print statements to the code to see where it crashes on LabTS.
5. The performance tests in Part 2 evaluates your pretrained model from the pickle file. For this to work, make sure you have actually committed your pickle file into the GitLab repository.
6. If you create your `part2_model.pickle` file on a machine with incompatible versions of libraries, then you might experience errors on `LabTS`. We recommend creating the pickle file in one of the lab machines, using the `/vol/lab/ml/mlvenv` virtual environment (or ensuring your local environment matches `requirements.txt`). If you want to work on

your own machine, make sure your python library versions match the ones in the virtual environment.

7. Note that LabTS does not have GPUs during testing. Depending on how you've saved your model, it might be looking for a GPU when loading, resulting in warnings or errors. If that happens, try training and saving your model using only your CPU.

8. Avoid committing Python environments, packages, compiled Python files, cache files, etc. to your GitLab repository. These take up unnecessary space and slow down `LabTS`. Use a `.gitignore` file to ignore any local files that you do not wish to be committed.

9. Note that the public `LabTS` tests will report your model performance on the same training data that has been provided to you. It is not suggested to directly tune for high performance on this test, as that would result in overfitting to the training data. The final private tests will measure the performance of your saved model on a separate held-out evaluation set.

Grading scheme

Part 1 and 2 will be evaluated only based on `LabTS` results.

There are 20 tests in total. 19 of these you are able to run by yourself at any time during development and get instant feedback about how the code is performing. There is one additional test that will be run on your code, which you will not see in advance. This test evaluates your part2 model performance on a previously unseen held-out dataset. **The main objective of the coursework is for you to get back feedback on your work.**

Maximum total for this coursework is 40 marks. The tests include:

- Part 1: 16 unit tests, checking for various functionalities, 1 mark each.
- Part 1: Performing gradient descent with the resulting model, 4 marks.
- Part 2: Testing basic Regressor calling, 6 marks.
- Part 2: Performance test on provided training data. Loads your saved model. Just need to surpass a given RMSE threshold. 6 marks.
- Part 2: Hidden performance test on held-out test data. Loads your saved model. Need to surpass a given RMSE threshold. 8 marks.

The performance tests are quite lenient. You don't need to engineer anything special – as long as your model works, it should pass these tests. You can separate a held-out development set yourself in order to check your performance on unseen data in advance.

A couple of tests already pass with the provided skeleton code. That is expected, some of these just check that the code doesn't crash. Your task is to complete the missing code while making sure all the tests still pass.